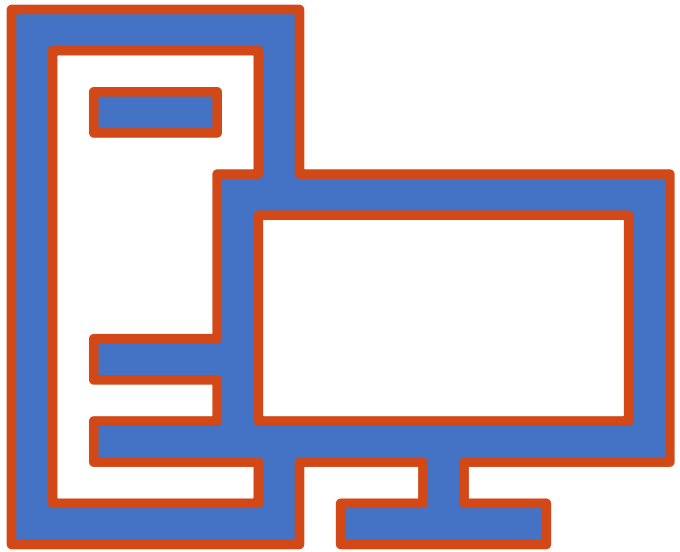

FUNCTIONAL PROGRAMMING & LAMBDA EXPRESSIONS

PROG1123

GOALS FOR TODAY

- **Understand more about anonymous functions**
- **Create and implement callback functions**
- **Meet fat arrow syntax!**
- **Become familiar with a few more JS array functions**

FUNCTION DECLARATION VS. EXPRESSIONS

- **//example of function declaration**

```
function greeting(n){
        console.log(`Hello ${n}!`);
}
//invoke greeting function
greeting('Marsha');
```

- **//example of function expression**

```
let greeting = function(){
        console.log(`Hello ${n}!`);
}
//invoke greeting function
greeting('Marsha');
```

Both of these functions look very similar, but they are treated very differently by the compiler!

Function Definitions are hoisted to the top in memory, and thus are available to be called by the program whenever.

Function expressions are placed in memory only when they are called.

ANONYMOUS FUNCTIONS

- Functions do not always need to have a name!
  - Why give a function a name if it is only called in one place?
- Benefits:
  - You avoid naming collisions across modules
  - There is no need to manage the name of the function in memory
- Drawbacks:
  - Anonymous functions are not accessible in memory until the point in the code where they are created
  - They can be hard to debug when their code raises an error

ANONYMOUS FUNCTIONS EXAMPLES

```
//in an assignment statement (function expression)
let greeting = function(name){
    console.log(`Hello there ${name}!`);
}
greeting("Dave");  // console logs "Hello there Dave!"
```

```
//as a callback function
window. addEventListener("load", function(){
        alert("window loaded!");
});
```

```
//as an Immediately Invoked Function Expression (IIFE)
(function(){
        console.log("hello there!");
})();
```

ANONYMOUS FUNCTIONS USED BY ARRAY PROTOTYPE METHODS: ALTERNATIVE TO FOR…OF

- One of the JS array prototype functions is **forEach().** It looks like this:

```
let courses = ["PROG1735", "PROG1198", "PROG1870"];
let output = "<ul>";

courses.forEach(function(c){
    output += `<li>${c}</li>`;
});  //each iteration, a function runs, and c holds the current value in the array

output += "<ul>";
document.write(output);
```

- NOTE: the anonymous function in the forEach() method is a callback function
- Anytime a function is passed as a parameter to another function, we call it a callback

FAT ARROW SYNTAX

- Fat arrow syntax removes the "noise" of function syntax. What is the noise?
  - The word function
  - The curly brackets and return keyword

- They are known in other languages as lambda expressions

```
let greeting = function(n){
        console.log(`Hello there ${n}!`);
 }

//converted to fat arrow syntax …
let greeting = (n)=>console.log(`Hello there ${n}!`);
greeting('Dave'); //logs Hello there Dave! To the console
```

MORE EXAMPLES OF FAT ARROW EXPRESSIONS

```
//example of function expression
let upperName = function(first, last){
    return `${first} ${last}`. toUpperCase();
}
//to convert to fat arrow syntax, take away function, curly brackets and return statement
let uppername = (first, last)=> `${first} ${last}`. toUpperCase();
```

PRACTICE

- Try writing this out in fat arrow syntax:

```
courses.forEach(function(c){
    output += `<li>${c}</li>`;
});
```

- Try your hand at the practice exercise that is attached with these slides